3. Symmetric Key Encryption using AES: Implement AES Algorithm to Perform Secure Encryption and Decryption of Data Using Python

1. Description

Advanced Encryption Standard (AES) is one of the world's most secure and widely adopted symmetric-key cryptographic algorithms. It was established by the National Institute of Standards and Technology (NIST) in 2001 and is extensively used for protecting confidential information in banking systems, cloud storage, military communications, e-commerce applications, healthcare systems, and government organizations.

AES belongs to the category of **Symmetric Key Cryptography**, where the same secret key is used for both encryption and decryption. Compared to the older DES algorithm, AES provides significantly stronger security, faster performance, and support for larger key sizes.

The AES algorithm works on fixed-size blocks of **128 bits (16 bytes)** and supports key lengths of:

- AES-128 (16-byte key)
- AES-192 (24-byte key)
- AES-256 (32-byte key)

The encryption process transforms readable information (Plaintext) into unreadable ciphertext using mathematical operations and a secret encryption key. The encrypted data can only be recovered using the same secret key.

In this practical, Python and the **PyCryptodome** cryptographic library are used to implement AES encryption and decryption. The program accepts plaintext from the user, generates a secure random encryption key, encrypts the message, and then decrypts it back to its original form.

This experiment demonstrates how confidential information can be securely protected during storage and transmission.

2. Objectives

After completing this practical, students will be able to:

1. Understand the concept of symmetric key cryptography.
2. Learn the working principle of the AES encryption algorithm.
3. Understand the role of secret keys in encryption and decryption.
4. Generate a secure AES encryption key.
5. Encrypt plaintext into ciphertext using AES.
6. Decrypt ciphertext back into the original plaintext.
7. Understand block cipher modes of operation.
8. Learn the purpose of Initialization Vector (IV).
9. Apply Python cryptographic libraries for secure communication.
10. Understand real-world applications of AES encryption.

3. Learning Outcomes

Upon successful completion of this practical, students will be able to:

- Explain symmetric encryption techniques.
- Differentiate plaintext, ciphertext, key, and IV.
- Generate secure cryptographic keys.
- Perform AES encryption using Python.
- Perform AES decryption correctly.
- Analyze encrypted hexadecimal output.
- Develop secure encryption-based applications.
- Apply AES for secure file and message protection.
- Understand the importance of cryptographic security.
- Implement secure data transmission techniques.

4. Software Requirements

- Python 3.x
- PyCryptodome Library
- VS Code / PyCharm / Jupyter Notebook

5. Installation

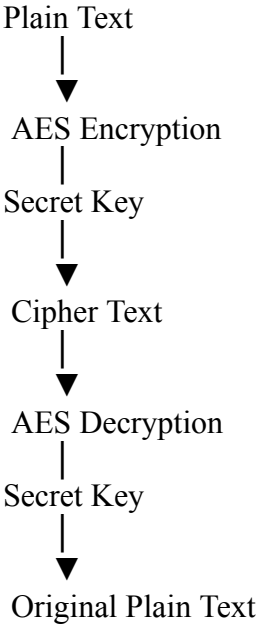
pip install pycryptodome

6. Theory

Symmetric Key Encryption

In symmetric encryption,

- Same key is used for encryption.
- Same key is used for decryption.



AES Encryption Process

AES performs multiple rounds consisting of

- Key Expansion
- SubBytes
- ShiftRows
- MixColumns
- AddRoundKey

The number of rounds depends on key size.

Key Size	Rounds
t	
t	
t	

7. Python Program (With Detailed Comments)

```
# ============================================================
# AES Encryption and Decryption using Python
# Google Colab Compatible
# ============================================================

# Install PyCryptodome (Run this cell once in Google Colab)
!pip -q install pycryptodome

# Import required libraries
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad

# ============================================================
# Step 1: Accept plaintext from the user
# ============================================================

plaintext = input("Enter the message to encrypt: ")

# Convert plaintext into bytes
data = plaintext.encode('utf-8')

# ============================================================
```

```
# Step 2: Generate a 128-bit (16-byte) AES secret key
# ============================================================

key = get_random_bytes(16)

# ============================================================
# Step 3: Create AES cipher object using CBC mode
# ============================================================

cipher = AES.new(key, AES.MODE_CBC)

# Save the Initialization Vector (IV)
iv = cipher.iv

# ============================================================
# Step 4: Encrypt the plaintext
# ============================================================

ciphertext = cipher.encrypt(pad(data, AES.block_size))

# ============================================================
# Step 5: Display Encryption Details
# ============================================================

print("\n========== Encryption ==========")
print("Original Plaintext :", plaintext)
print("Secret Key (Hex)   :", key.hex())
print("Initialization IV  :", iv.hex())
print("Cipher Text (Hex)  :", ciphertext.hex())

# ============================================================
# Step 6: Create a new cipher object for decryption
# ============================================================

decrypt_cipher = AES.new(key, AES.MODE_CBC, iv)

# ============================================================
# Step 7: Decrypt the ciphertext
# ============================================================
```

```
decrypted_data = unpad(
    decrypt_cipher.decrypt(ciphertext),
    AES.block_size
)

decrypted_text = decrypted_data.decode('utf-8')

# ============================================================
# Step 8: Display Decryption Result
# ============================================================

print("\n========== Decryption ==========")
print("Decrypted Text     :", decrypted_text)

# ============================================================
# Step 9: Verify Correctness
# ============================================================

if plaintext == decrypted_text:
    print("\nResult: Encryption and Decryption Successful.")
else:
    print("\nResult: Encryption and Decryption Failed.")
```

Output :

```
...     ---------------------------------------- 2.3/2.3 MB 25.8 MB/s eta 0:00:00
    Enter the message to encrypt: abcdefg

    ========== Encryption ==========
    Original Plaintext : abcdefg
    Secret Key (Hex)   : a0fd167424f1cd6530b5c6fd3d7020d3
    Initialization IV  : a14102848ae856205628e653a3bba607
    Cipher Text (Hex)  : 09834ce699b36af9f47338060dce619d

    ========== Decryption ==========
    Decrypted Text     : abcdefg

    Result: Encryption and Decryption Successful.
```

8. Step-by-Step Program Explanation

AES Encryption and Decryption using Python – Code Explanation

1. Install the Required Library

!pip -q install pycryptodome

Explanation

- Google Colab does not always include the **PyCryptodome** library.
- This command installs the cryptography library required to use the AES algorithm.
- The `-q` option installs the package in **quiet mode**, reducing unnecessary output.

2. Import Required Libraries

from Crypto.Cipher import AES

from Crypto.Random import get_random_bytes

from Crypto.Util.Padding import pad, unpad

Explanation

`from Crypto.Cipher import AES`

- Imports the **AES (Advanced Encryption Standard)** algorithm.
- Used to perform both encryption and decryption.

`from Crypto.Random import get_random_bytes`

- Generates a **cryptographically secure random key**.
- Produces unpredictable random bytes suitable for encryption.

`from Crypto.Util.Padding import pad, unpad`

- AES encrypts data in blocks of **16 bytes**.
- If the message is shorter than a multiple of 16 bytes, `pad()` adds extra bytes.
- After decryption, `unpad()` removes the added bytes to recover the original message.

3. Accept User Input

plaintext = input("Enter the message to encrypt: ")

Explanation

- Takes a message from the user.
- This is the **Plaintext**, which is readable data before encryption.

Example:

Enter the message to encrypt:

Cyber Security

4. Convert Text into Bytes

data = plaintext.encode('utf-8')

Explanation

- AES works only with **binary data (bytes)**.
- `encode('utf-8')` converts the text into bytes.

Example

Text:

Cyber Security

Bytes:

b'Cyber Security'

5. Generate AES Secret Key

key = get_random_bytes(16)

Explanation

- Creates a **16-byte (128-bit)** secret key.
- The same key is required for both encryption and decryption.
- Every program execution generates a different random key.

Example

Secret Key:

a4f89cb76d2ef19ab73e8d921ca1ef34

6. Create AES Cipher Object

cipher = AES.new(key, AES.MODE_CBC)

Explanation

This creates an AES encryption object.

- `key` → Secret encryption key.
- `AES.MODE_CBC` → Cipher Block Chaining mode.

CBC mode encrypts each block using information from the previous block, making encryption more secure than ECB mode.

7. Store Initialization Vector (IV)

iv = cipher.iv

Explanation

- CBC mode automatically generates a random **Initialization Vector (IV)**.
- IV improves security by ensuring that identical plaintext encrypted with the same key produces different ciphertext.
- The IV is required during decryption.

Example

IV:

d91fa8cb4f729aa6b923f70e823f04da

8. Encrypt the Plaintext

ciphertext = cipher.encrypt(pad(data, AES.block_size))

Explanation

`pad(data, AES.block_size)`

- Adds padding to the plaintext so its length becomes a multiple of **16 bytes**.

Example

Original:

Hello

After Padding:

Hello###########

(The actual padding follows the PKCS#7 standard, not # characters.)

`cipher.encrypt()`

- Encrypts the padded plaintext using the AES algorithm.
- Produces unreadable encrypted data called **Ciphertext**.

Example

Plaintext:

Cyber Security

Ciphertext:

f34a8cb1c98ae73...

9. Display Encryption Information

```
print("\n========== Encryption ==========")

print("Original Plaintext :", plaintext)

print("Secret Key (Hex)   :", key.hex())

print("Initialization IV  :", iv.hex())

print("Cipher Text (Hex)  :", ciphertext.hex())
```

Explanation

Displays:

- Original message
- Secret key
- Initialization Vector
- Encrypted ciphertext

`.hex()` converts binary bytes into hexadecimal format for easy reading.

Example

Secret Key:

4b9e2fd78ac3...

Ciphertext:

8fd4a3cb91fa...

10. Create Cipher for Decryption

```
decrypt_cipher = AES.new(key, AES.MODE_CBC, iv)
```

Explanation

Creates another AES object for decryption.

Uses:

- Same Secret Key
- Same CBC Mode
- Same Initialization Vector

If either the key or IV is different, decryption will fail.

11. Decrypt Ciphertext

```
decrypted_data = unpad(
    decrypt_cipher.decrypt(ciphertext),
    AES.block_size
)
```

Explanation

This statement performs three operations:

Step 1

```
decrypt_cipher.decrypt(ciphertext)
```

Decrypts the encrypted message.

Step 2

unpad(...)

Removes the padding added during encryption.

Step 3

Returns the original byte data.

Example

Encrypted:

9fd38ab91...

After Decryption:

b'Cyber Security'

12. Convert Bytes into Text

decrypted_text = decrypted_data.decode('utf-8')

Explanation

Converts the decrypted bytes back into a readable string.

Example

Bytes:

b'Cyber Security'

Text:

Cyber Security

13. Display Decrypted Message

```
print("\n========== Decryption ==========")

print("Decrypted Text     :", decrypted_text)
```

Explanation

Displays the original message recovered after decryption.

Output

```
========== Decryption ==========

Decrypted Text : Cyber Security
```

14. Verify the Result

```
if plaintext == decrypted_text:

   print("\nResult: Encryption and Decryption Successful.")

else:

   print("\nResult: Encryption and Decryption Failed.")
```

Explanation

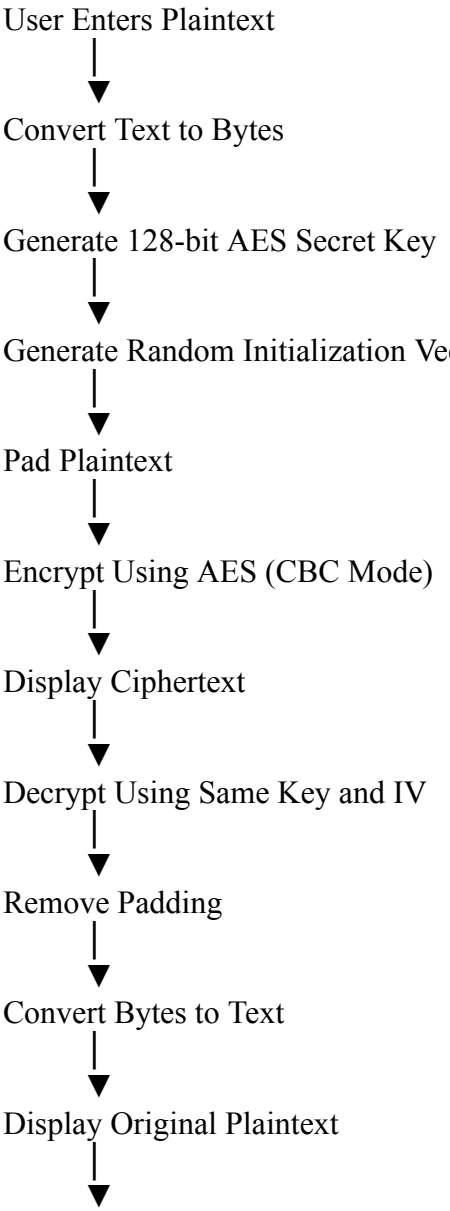
Compares the original plaintext with the decrypted text.

- If both are identical, the encryption and decryption process is successful.
- Otherwise, it indicates an error in the process.

Example Output

```
Result: Encryption and Decryption Successful.
```

Overall Program Flow



This program demonstrates the complete lifecycle of **AES symmetric-key encryption**: generating a secure key, encrypting plaintext into ciphertext, and decrypting it back to the original message using the same secret key and initialization vector.

9. Execution (Sample Run)

```
2.3/2.3 MB 25.8 MB/s eta 0:00:00
Enter the message to encrypt: abcdefg

========== Encryption ==========
Original Plaintext : abcdefg
Secret Key (Hex)   : a0fd167424f1cd6530b5c6fd3d7020d3
Initialization IV  : a14102848ae856205628e653a3bba607
Cipher Text (Hex)  : 09834ce699b36af9f47338060dce619d

========== Decryption ==========
```

```
Decrypted Text     : abcdefg

Result: Encryption and Decryption Successful.
```

10. Detailed Execution Flow

Start

Import AES Library

Read Plain Text

Generate Secret Key

Generate Initialization Vector

Create AES Cipher

Encrypt Plain Text

Display Cipher Text

Create AES Decryption Object

Decrypt Cipher Text

Display Original Message

End

11. Advantages of AES

- Highly secure encryption algorithm.
- Very fast execution.
- Supports 128-bit, 192-bit, and 256-bit keys.
- Resistant to brute-force attacks with appropriate key sizes.
- Efficient for both hardware and software implementations.
- Widely accepted international encryption standard.
- Used in secure internet communication (HTTPS), VPNs, cloud storage, and encrypted messaging.

12. Limitations

- Secure key distribution is essential; if the key is compromised, encrypted data can be decrypted.
- Encryption and decryption require the same secret key to be shared securely.
- Improper key management can reduce overall system security.
- Does not provide authentication by itself unless combined with authenticated modes (e.g., GCM) or message authentication mechanisms.

13. Applications

- Internet Banking
- ATM Transactions
- Secure File Storage
- Cloud Computing
- Healthcare Information Systems
- Military Communication
- Digital Wallets
- Password Managers
- Secure Email Systems
- Encrypted Messaging Applications
- Database Encryption
- Virtual Private Networks (VPNs)
- Wireless Network Security (WPA2/WPA3)

14. Result

Result:
The AES (Advanced Encryption Standard) algorithm was successfully implemented in Python using the PyCryptodome library. A secure 128-bit random secret key and an Initialization Vector (IV) were generated to encrypt the user-provided plaintext using AES in CBC mode. The plaintext was successfully transformed into encrypted ciphertext, demonstrating confidentiality. Using the same secret key and IV, the ciphertext was decrypted accurately to recover the original plaintext without any loss of information. The practical verified the correct implementation of symmetric key encryption and decryption, illustrating the effectiveness of AES in protecting sensitive data during storage and transmission.